M1 — Literature survey, agent design and project plan

Name: Jozef Jurcik **Team:** A Team **Specialisation:** Improviser (Navigation)

1 Literature survey

The Night Round combines two classical problems: exploring a graph that is revealed only as it is walked, and collecting items with a vehicle of limited capacity. Planning for every possible layout of unseen places takes too long, so planning and acting alternate [1].

Exploring an unknown graph. In online graph exploration, every vertex of a weighted graph must be visited, and the edges of a vertex are revealed on arrival [2]. Two simple rules have known guarantees. The nearest-neighbour rule always walks to the closest unvisited vertex, and its tour was shown to be at most $O(\log n)$ times as long as the best one [2, 3]. Depth-first search was shown to walk at most twice the number of vertices when all edges are equally long [4]. In simulations, walking to the closest border of the known space (the frontier) did not fall behind more elaborate strategies [5]. Both guarantees assume that the neighbours of a vertex are seen on arrival. In the Night Round, an untried exit shows only its length, not where it leads.

Carrying volumes home. The volumes add a second constraint, as the trolley has to return to the reading hall whenever it is full. Exploring with regular returns to the start was studied as piecemeal learning [6]. There each trip may walk only a limited distance. Depth-first search that breaks off to return may then be unable to resume, since the only known path back to where it stopped can be longer than one trip allows. The trolley is limited in load instead, so in the Night Round a return only costs the walk back. Once the map is known, the task is capacitated routing: trips from a depot with a load limit and as little total distance as possible [7]. A simple method for it is the savings heuristic [8]. The combination is known in robotics as foraging, in which robots search for objects and carry them to a collection point [9]. No source in this survey treats it for a single agent that keeps a map and is scored on distance.

Language models. Used alone, language models were found to plan poorly. GPT-4 solved about 12% of planning tasks on average [10] and performed poorly at mapping and navigation in text-game mazes [11]. Other work keeps a classical method in control: the model proposes and an external verifier checks [12], or the model only guides which frontier a planner tries next [13]. As the Night Round also scores tokens, the design in §2.1 was built on the nearest-neighbour rule, capacitated routing and the model as a checked fallback. A plain-prompt baseline, in which the course model plays alone [14], will show whether it is as weak.

2 Agent design

2.1 My agent

The problem. The first message gives the number of volumes (`total`), the capacity of the trolley and the exits of the hall with their lengths. On arrival, a place shows its id, kind (room or junction), volumes, exits with their lengths and the exit that leads back [14]. The agent is never shown where an untried exit leads or where the volumes are. The environment is thus partially observable [15, 16]. Within a run it is deterministic and static, so a stored map stays valid. The score counts volumes delivered, distance walked and tokens used, but not steps [17], so loading costs nothing. As `total` is known, exploring can stop once all volumes have been seen.

Approach. The agent is deliberative [16]: it keeps a map and plans on it in code, with shortest paths from NetworkX [18]. Until all volumes are seen, the first of rules 1 to 3 that applies is used.

1. In a room, as many volumes as fit are taken.
2. A full trolley, or one holding all remaining volumes, takes the shortest known path to the hall.
3. Otherwise the cheapest untried exit is walked. The cost is the known distance to the exit plus its length (the *nearest-exit rule*).
4. Once all volumes have been seen, the remaining trips are planned as capacitated routing, in which a room may be emptied over several trips [17]. All trip plans are tried when few rooms remain, the savings heuristic is used otherwise, and the plan decides every take.

Depth-first search was the alternative considered. As destinations are hidden, neither rule keeps its guarantee from §1: the nearest-exit rule may walk a long corridor only to arrive at a known place, and depth-first search has to try every exit. The nearest-exit rule was chosen for two reasons. First, the closest-frontier strategy held up in the simulations of §1. Second, after each unloading, depth-first search walks back to the place it discovered last, whereas the nearest-exit rule takes the cheapest exit, which is never farther away.

Volumes are loaded while exploring, because this saves a second walk to the room. The price is that a take cannot be undone [17], so a trolley filled early may force an extra trip, which will be logged. A language model as the driver was ruled out by the findings of §1 and by its token cost. It is kept only as a fallback for when the arena refuses the actions of the agent, and its proposal is sent only if the code confirms it [12].

Evaluation. A depth-first walker without a routing phase will be built first, as the simplest version and the alternative. The plain-prompt baseline [14], which gives the model the rules and the messages so far, will be run over several seeds, reported as mean and standard deviation [19]. All variants will be run on the same practice instances, some kept out of tuning. The design counts as working if every volume is delivered on every practice instance and the mean distance on the instances kept out of tuning is below that of the depth-first walker. If depth-first search walks less, it will be kept instead. The log will split the distance into exploring and carrying. If the agent falls short, the larger share is addressed first: rule 3 for exploring, rule 2 for carrying.

Where it meets the team. In the Overture, the Improviser is a service that the Conductor calls with a goal, such as fetching certain volumes or exploring towards a sign, and that returns the next action. Its map is kept in the shared store (§2.2). It needs to know which volumes are worth a place on the trolley (Musicologist) and which doors are open (Player). In return it reports the distance of a trip, to be weighed against the worth of the volumes.

2.2 The Ensemble

Dependencies. The parts depend on each other in the four ways named by Malone and Crowston [20]. There are two *shared resources*: the places on the trolley and the one connection to the arena. As *producer and consumer*, the Improviser delivers the volumes that the Musicologist can read only in the hall, and a door opened by the Player gives the Improviser a shortcut. *Simultaneity* is needed at a door, where a puzzle can be answered only while the trolley stands there [17]. Each question is a *task with subtasks*: choosing rooms, fetching, reading and answering.

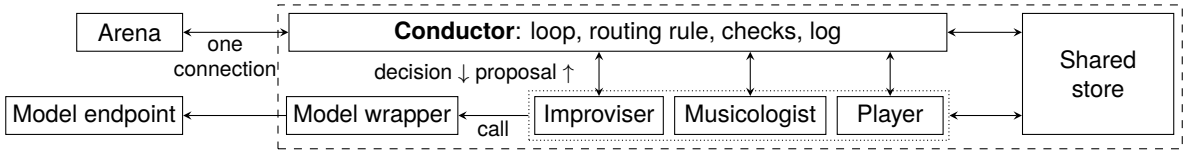


Figure 1: The container (dashed) with its three specialists (dotted). The Conductor and the specialists use the shared store. Only the specialists call the model, through the wrapper.

Coordination. The Ensemble is organised as a hierarchy [21]. At the top is the Conductor, written in plain code. It holds the connection, runs the loop, checks every action and hands each decision to one specialist (Fig. 1). It also keeps the shared store: the setlist texts, sent only

once, the map, the texts read and the costs. In a base quest, the first message names the quest and so the specialist. In the Overture the order of work is open, so a fixed rule settles where the trolley goes next: the Musicologist rates each candidate sign or volume, the Improviser gives its distance, and the Conductor takes the candidate with the most worth per distance.

The alternative considered was a blackboard [22], on which a control component selects which part writes next [21]. Its authors noted that such control "takes time and requires many fairly sophisticated mechanisms" and is justified where no explicit algorithm exists [22, p. 246]. Here a simple rule chooses the next step without using tokens, so only the shared store was taken over.

When a part fails, is slow or is wrong. If a specialist fails, the Conductor falls back on a default without tokens: the nearest exit or a skip. A slow model call is stopped before the endpoint's 300-second cut-off [14] and retried once, so a turn stays below the ten minutes of silence that end a run [17]. Against wrong answers, every proposal is first checked in code, as language models were found no better at verifying solutions than at producing them [12]. A failed proposal is returned once with the reason. If it fails again, the last well-formed one is sent, as a wrong answer costs no more than a skip [17], except on claims, which a team of three does not get.

Logging. The starter code logs every arena message and model call [14]. For each action, the log will add the deciding part, its reason, its tokens, the change in `spent` and the check result.

3 Project plan

3.1 The team's plan

By	What will be working	Who
16 Nov	Self-test passes, and fails when the prompt is broken	Merkl
20 Nov	Conductor shares: loop and routing (Jurcik), shared store (Merkl), logging and failure rules (Mukherjee)	one each
23 Nov	Each base quest plays a practice run via the Conductor	each owner
27 Nov	**First Overture run with all parts in one container**	Jurcik
30 Nov week	Rehearsal: image connects to course arena and endpoint	Jurcik
4 Dec	Overture end to end with failure rules, baselines measured	Mukherjee
8 Dec	Image frozen, all dependencies inside, reflections drafted	Mukherjee
10 Dec	M2 handed in	each

Each member owns one Conductor share. Open rows are checked weekly and work is moved.

3.2 My plan

16 Nov: parser, map store and depth-first baseline. 20 Nov: the Conductor share. 23 Nov: nearest-exit rule, plain-prompt baseline and measurement script. 27 Nov: service interface for the Overture. 1 Dec: routing phase. 4 Dec: model fallback and comparison of the variants. Two steps are most likely to slip. The Conductor share blocks both teammates and will therefore be done before the nearest-exit rule. If the hand-over to the Overture slips, the minimal "next action" call will be kept and exploring by signs will be moved to after M2.

References

[1] S. Koenig, "Agent-centered search," *AI Magazine*, vol. 22, no. 4, pp. 109–131, 2001. DOI: 10.1609/aimag.v22i4.1596

[2] F. Eberle, A. Lindermayr, N. Megow, L. Nölke, and J. Schlöter, "Robustification of online graph exploration methods," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, 2022, pp. 9732–9740. DOI: 10.1609/aaai.v36i9.21208

[3] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis II, "An analysis of several heuristics for the traveling salesman problem," *SIAM Journal on Computing*, vol. 6, no. 3, pp. 563–581, 1977. DOI: 10.1137/0206041

[4] S. Koenig, C. Tovey, and Y. Smirnov, "Performance bounds for planning in unknown terrain," *Artificial Intelligence*, vol. 147, no. 1–2, pp. 253–279, 2003. DOI: 10.1016/S0004-3702(03)00062-6

[5] D. Holz, N. Basilico, F. Amigoni, and S. Behnke, "Evaluating the efficiency of frontier-based exploration strategies," in *Proceedings of ISR 2010 (41st International Symposium on Robotics) and ROBOTIK 2010 (6th German Conference on Robotics)*, Berlin: VDE Verlag, 2010, pp. 36–43. [Online]. Available: https://www.ais.uni-bonn.de/~holz/papers/holz_2010_isr.pdf

[6] M. Betke, R. L. Rivest, and M. Singh, "Piecemeal learning of an unknown environment," *Machine Learning*, vol. 18, no. 2–3, pp. 231–254, 1995. DOI: 10.1007/BF00993411

[7] M. Haimovich and A. H. G. Rinnooy Kan, "Bounds and heuristics for capacitated routing problems," *Mathematics of Operations Research*, vol. 10, no. 4, pp. 527–542, 1985. DOI: 10.1287/moor.10.4.527

[8] G. Clarke and J. W. Wright, "Scheduling of vehicles from a central depot to a number of delivery points," *Operations Research*, vol. 12, no. 4, pp. 568–581, 1964. DOI: 10.1287/opre.12.4.568

[9] A. F. T. Winfield, "Foraging robots," in *Encyclopedia of Complexity and Systems Science*, R. A. Meyers, Ed., New York, NY: Springer, 2009, pp. 3682–3700. DOI: 10.1007/978-0-387-30440-3_217

[10] K. Valmeekam, M. Marquez, S. Sreedharan, and S. Kambhampati, "On the planning abilities of large language models: A critical investigation," in *Advances in Neural Information Processing Systems 36*, Curran Associates, 2023, pp. 75 993–76 005. DOI: 10.52202/075280-3320

[11] P. Ding et al., "MANGO: A benchmark for evaluating mapping and navigation abilities of large language models," in *First Conference on Language Modeling (COLM 2024)*, 2024. arXiv: 2403.19913.

[12] S. Kambhampati et al., "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, PMLR, 2024, pp. 22 895–22 907. arXiv: 2402.01817.

[13] D. Shah, M. R. Equi, B. Osiński, F. Xia, B. Ichter, and S. Levine, "Navigation with large language models: Semantic guesswork as a heuristic for planning," in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, PMLR, 2023, pp. 2683–2699. arXiv: 2310.10103.

[14] J. Schaumeier, *The Archive Games*, Lecture slides, Session 4, IN2106 Multi-Agent AI Lab, Technical University of Munich, Nov. 9, 2026.

[15] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., global ed. Pearson, 2021. [Online]. Available: https://aima.cs.berkeley.edu/global-index.html

[16] J. Schaumeier, *How does one agent decide?* Lecture slides, Session 1, IN2106 Multi-Agent AI Lab, Technical University of Munich, Oct. 19, 2026.

[17] IN2106 Multi-Agent AI Lab, *The Archive Games: Rules and API reference*, Course document, IN2106 Multi-Agent AI Lab, Technical University of Munich, Nov. 9, 2026.

[18] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, G. Varoquaux, T. Vaught, and J. Millman, Eds., 2008, pp. 11–15. DOI: 10.25080/TCWV9851

[19] A. Hochlehnert, H. Bhatnagar, V. Udandarao, S. Albanie, A. Prabhu, and M. Bethge, "A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility," in *Second Conference on Language Modeling (COLM 2025)*, 2025. arXiv: 2504.07086.

[20] T. W. Malone and K. Crowston, "The interdisciplinary study of coordination," *ACM Computing Surveys*, vol. 26, no. 1, pp. 87–119, 1994. DOI: 10.1145/174666.174668

[21] J. Schaumeier, *How do several agents decide together?* Lecture slides, Session 2, IN2106 Multi-Agent AI Lab, Technical University of Munich, Oct. 26, 2026.

[22] L. D. Erman, F. Hayes-Roth, V. R. Lesser, and D. R. Reddy, "The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty," *ACM Computing Surveys*, vol. 12, no. 2, pp. 213–253, 1980. DOI: 10.1145/356810.356816

Use of AI

TODO